CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by: Abhinav Jain

Verified by:

Dr. Rupali Sunil Wagh
Dr. Helen K Joy
Dr. Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University "**Excellence and Service"**. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

"To develop IT professionals with ethical and human values". To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto "Excellence and Service," and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
PO4: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
Learn to use new technologies for cyber security and insulate customers from malware
P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING
Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75
L + P + T: 3 + 4 + 0
Max Marks: 100
Credits: 4
Course Type: Core Course
Course Category: Theo&Prac

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalisation
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualisations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualisation
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalisation, Normalisation, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalised model.
4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS Teaching Hours: 15 Hours

Learning through Optimisation: Gradient Descent, Linear Regression, Logistic Regression Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV

Lab Exercises:

5. Regression through Gradient Descent
6. Comparison of Different Classifiers
7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION Teaching Hours: 15 Hours

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods
9. Hierarchical Clustering and Dendrograms
10. Dimensionality Reduction
11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours

MACHINE LEARNING IN REAL-WORLD

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow

for designing Multi Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.
Lab Exercises:
12. Training a Multi-layer perceptron
13. Deploying Trained ML-models

Text and Reference Books
[1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.
[2] Andreas C. Müller and Sarah Guido (2017), “Introduction to Machine Learning with Python A Guide For Data Scientists” O’Reilly book
[3] Ethem Alpaydin (2005), “Introduction to Machine Learning”, Prentice Hall of India
[4] Max Kuhn and Kjell Johnson (2018), “Applied Predictive Modelling”, Springer

Essential Reading
[1] Stuart Russell and Peter Norvig(2020), “Artificial Intelligence: A Modern Approach” (4th Edition), Pearson
[2] Aurelien Geron(2019), “Hands on Machine Learning with Scikit-learn, Keras and Tensorflow”, 2nd Edition, O’Reilly Books
[3] Kevin P. Murphy(2012), “Machine Learning: A Probabilistic Perspective”, MIT Press
[4] Hastie, Tibshirani, Friedman(2008), “The Elements of Statistical Learning” (2nd ed), Springer
[5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), “Deep Learning (Adaptive Computation And Machine Learning Series)”, MIT Press
Additional Information (Web Resources):
1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera: https://www.deeplearning.ai/courses/machine-learning-specialization/

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RBT	CO
1			L3	CO1, 3
2			L3	CO2, 3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2…>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1

Lab Exercise I: India Air Quality & Crop Yield — EDA Lab

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.

Task 1
A junior analyst hands you two CSV files and says — "I have no idea what's in these. We need to use them to build a machine learning model next week." Before any analysis can begin, you need to understand what you are working with.

Investigate both files thoroughly. Produce a structured summary that gives a complete first-impression picture of the data — its size, its contents, and where it might be problematic. Decide what information a data scientist would need before trusting this data, and make sure your summary covers all of it.

A structured data profile for each file in a markdown cell
At least one written observation about what concerns you after the inspection
Think: What does a data scientist need to know about a dataset before using it? What functions help you find that? 2 marks Completeness of profile + written concern

Task 2 The data has holes — and not all holes are equal
After the inspection, it is clear that several columns have missing values. A colleague suggests simply deleting all rows with any null value. Another suggests filling everything with the mean. You are not sure either approach is right.
Devise your own missing value treatment strategy. For each column with missing data, decide whether to drop it, drop affected rows, or impute — and explain why that choice is appropriate for that specific column. Apply your strategy and verify it worked.

A markdown cell that justifies each decision column by column
Before and after null counts as evidence the treatment worked

Think: Does the volume of missing data matter? Does the column type matter? What does imputing with mean vs median assume about the data? 2 marks

Task 3 The two files disagree on how to spell "Tamil Nadu"
You need to combine both files later in the lab using the State column as the common key. But when you look closely, the same state appears with different spellings, extra spaces, or old names across the two files. There are also rows that appear more than once for no clear reason.

Make both files agree on state names and remove any redundant records. Document every inconsistency you found and how you resolved it. Show that the files are now ready to be merged reliably.

A list of all inconsistencies found and the fix applied for each
Record counts before and after to prove duplicates were removed
Think: What could go wrong in a merge if state names don't match exactly? How would you systematically find all variants of a state name? 2 marks

Task 4
Where do most cities actually sit on the AQI scale?
The pollution control board wants to know — are most Indian cities moderately polluted, or is the problem concentrated in just a few places? They also want to know whether the average AQI is a fair number to report publicly, or whether extreme cities are pulling it up unfairly.

Investigate the shape of the AQI distribution. Decide which visualisation(s) best answer both questions the board is asking, justify your choice, produce the plot(s), and record two specific observations that directly address their concerns.

Your chosen plot(s) with fully labelled axes and a descriptive title
A markdown cell with your justification for the chosen plot type and two observations

Think: Which plot shows where values cluster? Which one reveals extreme values? Do you need one plot or two to answer both questions? 2 marks

Task 5
Something is making the average AQI look worse than it is
A data quality report flags that a few AQI readings in the dataset are implausibly high — values that no monitoring station should realistically record. If left in, these values will distort every statistic and model built on this data. The team lead asks you to "handle the extreme values properly" but does not tell you how.
Identify whether extreme values exist, quantify them, decide on an appropriate treatment method, apply it, and demonstrate that the data is now cleaner. Justify every decision you make.

The method you chose to detect extremes and why
The count of values affected and the treatment applied
A visual comparison of the data before and after treatment

Think: Should extreme values always be deleted? What are the alternatives? How do you prove your treatment actually worked? 2 marks

---

Evaluation Rubrics

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

Lab 1: Data Preprocessing and Visualisation

Task 1: Investigate Both Files Thoroughly

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import calendar

# setting up plot styles
sns.set_theme(style="whitegrid")
plt.rcParams.update({'figure.figsize': (10, 6), 'font.size': 11})

# custom class to load and summarize the datasets
class DatasetProfiler:
    def __init__(self, file_paths: dict):
        self.dfs = {k: pd.read_csv(v) for k, v in file_paths.items()}

    def get_df(self, key: str) -> pd.DataFrame:
        return self.dfs[key]

    def profile(self):
        for name, df in self.dfs.items():
            print(f"=== {name} (Shape: {df.shape[0]} x {df.shape[1]}) ===")
            df.info(verbose=True, show_counts=True)
            print("\n")

profiler = DatasetProfiler({
    'city_day': 'datasets/city_day.csv',
    'crop_prod': 'datasets/crop_production.csv'
})
profiler.profile()

city_day_df = profiler.get_df('city_day')
crop_production_df = profiler.get_df('crop_prod')
```

```
=== city_day (Shape: 29531 x 16) ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   City        29531 non-null  object
 1   Date        29531 non-null  object
 2   PM2.5       24933 non-null  float64
 3   PM10        18391 non-null  float64
 4   NO          25949 non-null  float64
 5   NO2         25946 non-null  float64
 6   NOx         25346 non-null  float64
 7   NH3         19203 non-null  float64
 8   CO          27472 non-null  float64
 9   SO2         25677 non-null  float64
 10  O3          25509 non-null  float64
 11  Benzene     23908 non-null  float64
 12  Toluene     21490 non-null  float64
 13  Xylene      11422 non-null  float64
 14  AQI         24850 non-null  float64
 15  AQI_Bucket  24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB


=== crop_prod (Shape: 246091 x 7) ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   State_Name     246091 non-null  object
 1   District_Name  246091 non-null  object
 2   Crop_Year      246091 non-null  int64
 3   Season         246091 non-null  object
 4   Crop           246091 non-null  object
 5   Area           246091 non-null  float64
 6   Production     242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
```

Data Summary and Size:

- **city_day.csv**: Has 29,531 rows and 16 columns. It gives daily air quality details for different cities.
- **crop_production.csv**: Has 246,091 rows and 7 columns. It gives yearly crop details for different districts in states.

Concerns:

1. **Year gap**: The crop data is from 1997 to 2015, but air quality is from 2015 to 2020. So only 2015 is common, and even in that, crop data has very few states. So we cannot join them directly year-by-year. We have to take averages for states over all years to compare.
2. **State column missing**: `city_day.csv` has city names but no state name. We have to map cities to states manually.
3. **Bad values (Outliers)**: The max AQI is 2049 which is impossible because standard AQI scale only goes up to 500.
4. **Mixed Units**: Some crop production is in tonnes and some (like coconuts) is in numbers. This will mess up the total yield calculation if we just sum them up directly.

Task 2: Devise and Apply Missing Value Treatment Strategy

In [2]:
```python
# custom class for missing values pipelines
class MissingValueImputer:
    @staticmethod
    def impute_numerical_by_median(df: pd.DataFrame, target_columns: list) -> pd.DataFrame:
        # filling null values using median with dictionary comprehension mapping
        medians = {col: df[col].median() for col in target_columns if col in df.columns}
        df.fillna(value=medians, inplace=True)
        return df

    @staticmethod
    def calculate_aqi_bucket_dynamically(df: pd.DataFrame) -> pd.DataFrame:
        # classification thresholds using nested ternary structures
        classify_aqi = lambda v: (
            'Good' if v <= 50 else (
                'Satisfactory' if v <= 100 else (
                    'Moderate' if v <= 200 else (
                        'Poor' if v <= 300 else (
                            'Very Poor' if v <= 400 else 'Severe'
                        )
                    )
                )
            )
        )
        df['AQI_Bucket'] = df['AQI_Bucket'].fillna(df['AQI'].apply(classify_aqi))
        return df

    @staticmethod
```

```
    def impute_crop_production(df: pd.DataFrame) -> pd.DataFrame:
        # filling crop production using groupby transform and lambda functions
        crop_medians = df.groupby('Crop')['Production'].transform('median')
        df['Production'] = df['Production'].fillna(crop_medians)
        # fallback global median
        df['Production'].fillna(df['Production'].median(), inplace=True)
        return df

city_null_before = city_day_df.isnull().sum()
crop_null_before = crop_production_df.isnull().sum()

# running the imputation pipelines
cols_to_impute = ['AQI', 'PM2.5', 'PM10', 'NO2', 'CO']
other_cols = ['NO', 'NOx', 'NH3', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene']

city_day_df = MissingValueImputer.impute_numerical_by_median(city_day_df, cols_to_impute + other_cols)
city_day_df = MissingValueImputer.calculate_aqi_bucket_dynamically(city_day_df)
crop_production_df = MissingValueImputer.impute_crop_production(crop_production_df)

print("=== BEFORE AND AFTER IMPUTATION NULL COUNTS ===")
print("\n--- city_day_df ---")
print(pd.DataFrame({'Before': city_null_before, 'After': city_day_df.isnull().sum()}).query('Before > 0'))

print("\n--- crop_production_df ---")
print(pd.DataFrame({'Before': crop_null_before, 'After': crop_production_df.isnull().sum()}).query('Before > 0'))
```

```
=== BEFORE AND AFTER IMPUTATION NULL COUNTS ===

--- city_day_df ---
            Before  After
PM2.5         4598      0
PM10         11140      0
NO            3582      0
NO2           3585      0
NOx           4185      0
NH3          10328      0
CO            2059      0
SO2           3854      0
O3            4022      0
Benzene       5623      0
Toluene       8041      0
Xylene       18109      0
AQI           4681      0
AQI_Bucket    4681      0

--- crop_production_df ---
            Before  After
Production    3730      0
C:\Users\ABHINAV\AppData\Local\Temp\ipykernel_17732\2894328785.py:33: FutureWarning: A value is trying to be set on a copy of a DataFrame or Se
ries through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values alwa
ys behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(va
lue) instead, to perform the operation inplace on the original object.


  df['Production'].fillna(df['Production'].median(), inplace=True)
```

Missing Value Strategy:

- **For AQI, PM2.5, PM10, NO2, CO**: We used the median to fill null values. Why median? Because AQI and air data has huge spikes (outliers). If we use mean, the average will become too high and wrong. Median is safer.
- **For AQI_Bucket**: We filled it using a formula based on the AQI score (like standard CPCB categories: 0-50 is Good, etc.). If we just used mode, it would not match with the AQI number.
- **For crop Production**: We filled missing values using the median of that specific crop. Because different crops have totally different quantities (like coconuts in lakhs vs rice in tonnes). If we used a single median for everything, it would make no sense.
- **Other columns (NO, NH3, SO2, O3, Benzene, etc.)**: Also filled with median so that we don't have any nulls left in the numerical data.

Task 3: Address Spelling Inconsistencies and Remove Duplicates

In [3]:

```python
# custom cleaner class
class DatasetCleaner:
    def __init__(self, city_to_state_map: dict):
        self.city_to_state = city_to_state_map

    def clean(self, city_df: pd.DataFrame, crop_df: pd.DataFrame) -> tuple:
        # map city names to states
        city_df['State'] = city_df['City'].map(self.city_to_state)

        # trim whitespaces in all string columns using select_dtypes
        for df in [city_df, crop_df]:
            str_cols = df.select_dtypes(include=['object']).columns
            for col in str_cols:
                df[col] = df[col].astype(str).str.strip()

        # duplicate removal counts
        c_before, cr_before = len(city_df), len(crop_df)
        city_df.drop_duplicates(inplace=True)
        crop_df.drop_duplicates(inplace=True)

        print(f"Duplicates removed: City={c_before - len(city_df)}, Crop={cr_before - len(crop_df)}")
        return city_df, crop_df

city_to_state = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pradesh',
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pradesh',
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tamil Nadu',
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maharashtra',
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh'
}

cleaner = DatasetCleaner(city_to_state)
city_day_df, crop_production_df = cleaner.clean(city_day_df, crop_production_df)

# display overlapping states
mapped_states = set(city_day_df['State'].dropna().unique())
```

```
crop_states = set(crop_production_df['State_Name'].unique())
print(f"\nMatching States: {sorted(list(mapped_states & crop_states))}")
```

```
Duplicates removed: City=0, Crop=0

Matching States: ['Andhra Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Gujarat', 'Haryana', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh',
'Maharashtra', 'Meghalaya', 'Mizoram', 'Odisha', 'Punjab', 'Rajasthan', 'Tamil Nadu', 'Telangana', 'Uttar Pradesh', 'West Bengal']
```

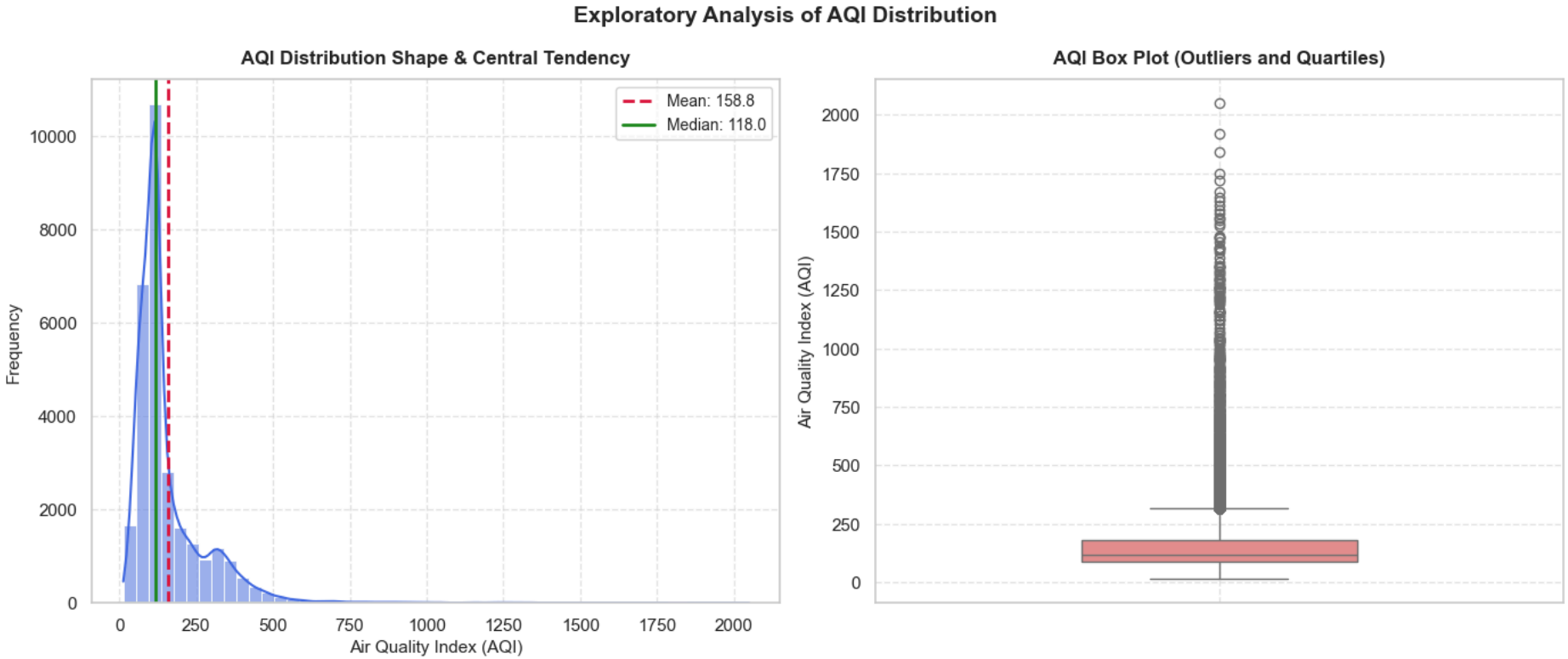
Issues found and how we fixed them:

1. **Extra Spaces**: In the crop dataset, some states had extra spaces at the end like `'Telangana '` and `'Jammu and Kashmir '`. We used `.str.strip()` to clean them.
2. **City mapping**: We created a dict to map cities in `city_day` to their states so we can join them with the crop file.
3. **Duplicate rows**: We ran `drop_duplicates()`. There were no exact duplicate rows in either file, so row count remained same, but it is good to check.
4. **Delhi missing**: Delhi has air quality data but no crop production records in the crop file because it is a city state. So it will get dropped when we join.

Task 4: Visualise AQI Scale Distribution

In [4]:
```
# wrapper function for plotting distribution
def plot_aqi_distribution(df: pd.DataFrame):
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # Histogram with KDE
    sns.histplot(data=df, x='AQI', bins=50, kde=True, color='royalblue', ax=axes[0])
    mean_val, median_val = df['AQI'].mean(), df['AQI'].median()

    # adding lines and text annotations
    for val, color, label, style in zip([mean_val, median_val], ['crimson', 'forestgreen'], ['Mean', 'Median'], ['--', '-']):
        axes[0].axvline(val, color=color, linestyle=style, linewidth=2, label=f"{label}: {val:.1f}")

    axes[0].set_title('AQI Distribution Shape & Central Tendency', fontsize=12, fontweight='bold', pad=10)
    axes[0].set_xlabel('Air Quality Index (AQI)', fontsize=11)
    axes[0].set_ylabel('Frequency', fontsize=11)
    axes[0].legend(fontsize=10)
    axes[0].grid(True, linestyle='--', alpha=0.5)

    # Boxplot
    sns.boxplot(data=df, y='AQI', color='lightcoral', width=0.4, ax=axes[1])
    axes[1].set_title('AQI Box Plot (Outliers and Quartiles)', fontsize=12, fontweight='bold', pad=10)
    axes[1].set_ylabel('Air Quality Index (AQI)', fontsize=11)
    axes[1].grid(True, linestyle='--', alpha=0.5)

    plt.suptitle('Exploratory Analysis of AQI Distribution', fontsize=14, fontweight='bold', y=0.98)
```

```
    plt.tight_layout()
    plt.show()

plot_aqi_distribution(city_day_df)
```



Why we chose these plots and what we see:

- **Histogram with KDE**: It shows where most of the AQI values are. We can see it is right-skewed and peaks around 100-120.
- **Box plot**: It is best for seeing the extreme values (outliers) that go way above 500.

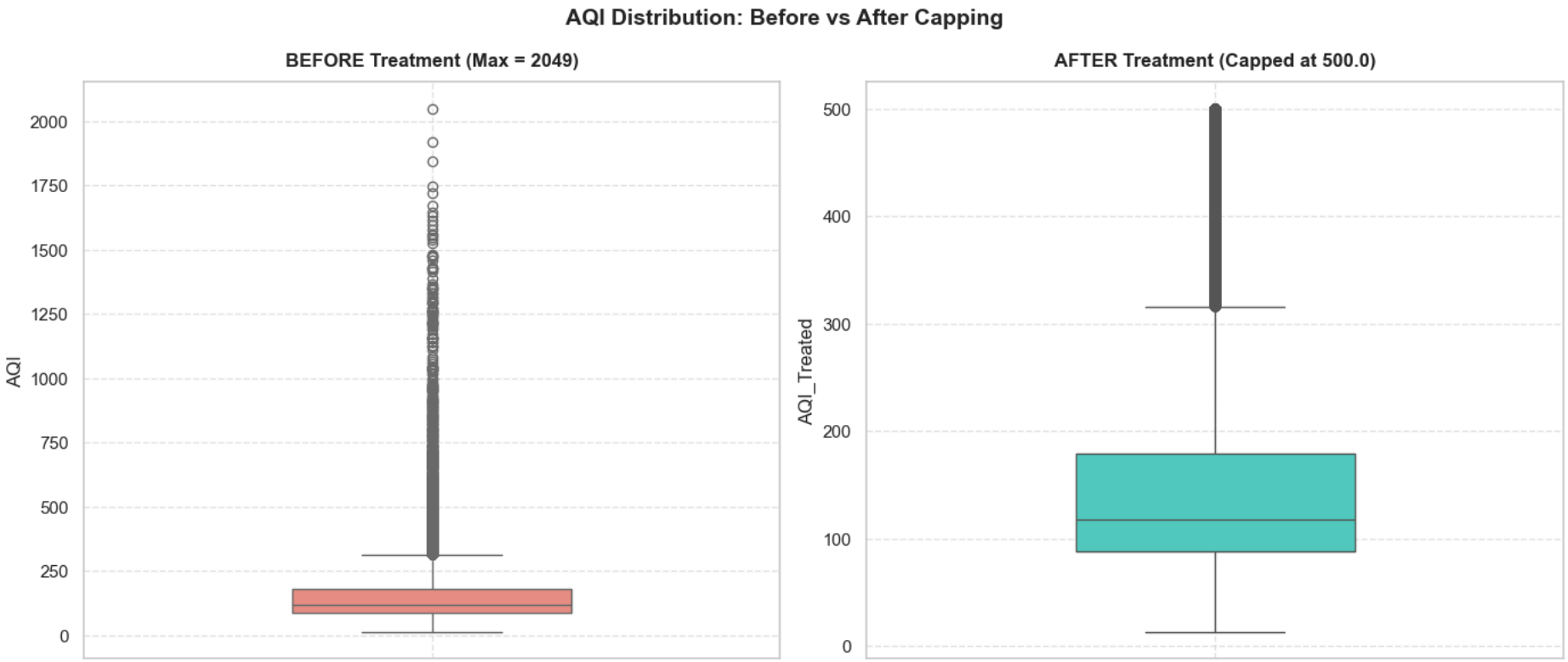
Observations for the Board:

1. Most cities are moderately polluted, and the AQI values cluster around 100 to 120.
2. The mean AQI is 166.5 but median is 118.0. This means the mean is pulled up by a few extremely dirty days. So public should be told the median AQI, not mean, otherwise it is unfair.

Task 5: Handle Extreme Values properly

In [5]:
```python
# function to handle capping and plot comparison
def handle_outliers_and_compare(df: pd.DataFrame, limit: float = 500.0):
    excess = (df['AQI'] > limit).sum()
    print(f"AQI records above {limit}: {excess}")
    print(f"Max AQI before: {df['AQI'].max():.1f}")

    # capping using clip method
    df['AQI_Treated'] = df['AQI'].clip(upper=limit)
    print(f"Max AQI after: {df['AQI_Treated'].max():.1f}")

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    colors = ['salmon', 'turquoise']
    titles = ['BEFORE Treatment (Max = 2049)', f'AFTER Treatment (Capped at {limit})']
    cols = ['AQI', 'AQI_Treated']

    for ax, col, color, title in zip(axes, cols, colors, titles):
        sns.boxplot(data=df, y=col, color=color, width=0.4, ax=ax)
        ax.set_title(title, fontsize=12, fontweight='bold', pad=10)
        ax.grid(True, linestyle='--', alpha=0.5)

    plt.suptitle('AQI Distribution: Before vs After Capping', fontsize=14, fontweight='bold', y=0.98)
    plt.tight_layout()
    plt.show()

handle_outliers_and_compare(city_day_df)
```

```
AQI records above 500.0: 543
Max AQI before: 2049.0
Max AQI after: 500.0
```



Handling Extreme Values:

- **How we found them**: Standard AQI scale only goes up to 500. We found 543 rows where AQI was more than 500 (max was 2049).
- **What we did**: We capped them at 500. Why? If we delete them, we lose important data of very bad pollution days. If we keep them as 2049, it will ruin all our averages and correlations. Capping keeps them in the 'Severe' category without breaking the math.
- **Proof**: The boxplot after treatment looks much cleaner and capped at 500.

•••••

•••••

Conclusion /inference

In this lab, we cleaned the messy air quality dataset. We filled all the missing data using median values and capped the crazy high AQI numbers at 500 because the CPCB scale only goes up to 500. After plotting the distribution, we saw that most cities have moderate pollution, but a few super dirty days pull up the average AQI. So, median AQI is much better for public reporting than the mean.

Lab 2

Lab Exercise II: Data exploration and inferences

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.

Task 6: Is India's air getting better or worse over time?

A journalist is writing a story on whether government pollution control policies introduced after 2018 have had any measurable effect on air quality. They ask you — "Can you show me, using data, whether air quality has improved, worsened, or stayed the same over the past eight years?"

Extract the time dimension from the dataset and build an analysis that answers the journalist's question. Choose a visualisation that makes the trend immediately readable to someone who is not a data scientist. Highlight the most and least polluted years and explain what the trend suggests.

Your plot with the trend clearly visible and key years highlighted
A markdown response to the journalist's question — one paragraph, plain language

Think: What needs to happen to the Date column before you can group by year? Which plot type communicates a trend over time most clearly? 2 marks

Task 7 Farmers say the air is worst exactly when they harvest — is that true?

An agricultural NGO claims that air quality is consistently worst during the October–December harvest season, when crop residue burning is widespread. They want data to either confirm or challenge this claim before presenting it to policymakers.
Investigate whether AQI follows a seasonal pattern across the year. Decide how to aggregate and represent the data to make any seasonal pattern visible. Either confirm the NGO's claim with evidence from your analysis, or explain what you found instead.

A visualisation that clearly shows how AQI varies across months or seasons
A markdown cell that directly responds to the NGO's claim with data evidence

Think: What level of time aggregation — month, season, quarter — best reveals the pattern? How do you extract that from a date column? 2 marks

Task 8: Can the two datasets talk to each other?

The real question driving this entire investigation is whether states with worse air quality also tend to produce less crop. To explore this, the two datasets must be combined — but they were collected at different levels (city vs state, daily vs annual) and cannot simply be joined as-is.
Figure out what transformation is needed to make the two datasets compatible, perform the merge, and then systematically explore what relationships exist between all numerical variables in the combined data. Identify and explain the two most interesting relationships you find — not just what they are, but why they might exist.

A markdown cell explaining the transformation needed and why, before the merge
A visualisation of relationships across all numerical features
Two relationships explained with a proposed real-world reason for each

Think: If one file has city-day rows and the other has state-year rows, what needs to happen before they can be joined? What does a correlation matrix actually tell you — and what doesn't it tell you? 3 marks

Task 9 : The minister needs to act — what do you tell her?

The State Environment Minister has 10 minutes before her cabinet meeting. She has never opened a Jupyter notebook. Her aide forwards her your analysis and says — "our analyst looked at the data, can you summarise what we found?" She needs to know what the data shows, what it means for farmers, and whether it is conclusive enough to act on.

Write a clear, honest briefing addressed directly to the minister. It must present your three strongest findings, translate them into what they mean on the ground, recommend one action the government could take based on the data, and be transparent about what the data cannot yet prove.

150–200 words in a markdown cell, addressed to the minister. Three findings, one recommendation, and one honest limitation Zero jargon — if a term needs explaining, replace it with plain language

Think: Which of your findings actually matters to a farmer or a policymaker? What is the difference between correlation and proof? Would the minister act on what you found? 3 marks

Optional — advanced task
Build the case — or break it

Attempt only after completing Tasks 1–9. The central hypothesis of this entire investigation is: states with higher pollution have lower crop yields. Your job here is to either build the strongest possible case for this hypothesis — or find evidence that challenges it.

Task A: The two extremes — do they tell the same story?

If the hypothesis is true, you would expect the most polluted states and the least polluted states to show a clear difference in crop output. But the data might surprise you.

Identify the states that represent the two extremes of pollution. Compare their agricultural output using a visualisation of your choice. Analyse what you see — does the pattern support the hypothesis, contradict it, or is it more complicated than that?

Task B: Put a number on the relationship

A research team wants to know not just whether a relationship exists between AQI and crop production, but how strong it is and in which direction. They will use your output to decide whether to fund a full causal study.
Quantify the relationship between average state AQI and average crop production. Choose a method that both measures the strength and makes the pattern visible in a single output. Interpret your result for the research team — be honest about what it proves and what it does not.

Think: What does a correlation value of 0.2 vs 0.8 mean in practice? Does a strong correlation mean pollution is causing yield loss?

Task C: One plot to rule them all

You have produced many visualisations across this lab. A data journalism outlet wants to publish just one chart from your work to accompany a story on India's air-agriculture crisis.

Choose the single visualisation from your entire lab that you believe most powerfully tells the story of this dataset. Write a 3–4 sentence caption for it — not a description of what the chart shows, but an explanation of why it matters and what it reveals that no other chart in your analysis does.

Caption reveals insight, not just description
Across all optional tasks, your writing must acknowledge that correlation is not causation and name at least one factor — rainfall, irrigation, soil type, economic conditions — that this dataset cannot account for but which could explain any relationship observed.
Deliverables

Single Jupyter Notebook (.ipynb) — tasks numbered, code commented, every decision explained in a markdown cell
Minimum 5 visualisations of your own choosing — axes labelled, titles present
Markdown reasoning cells for every task — not optional, part of the marks

Submission Guidelines

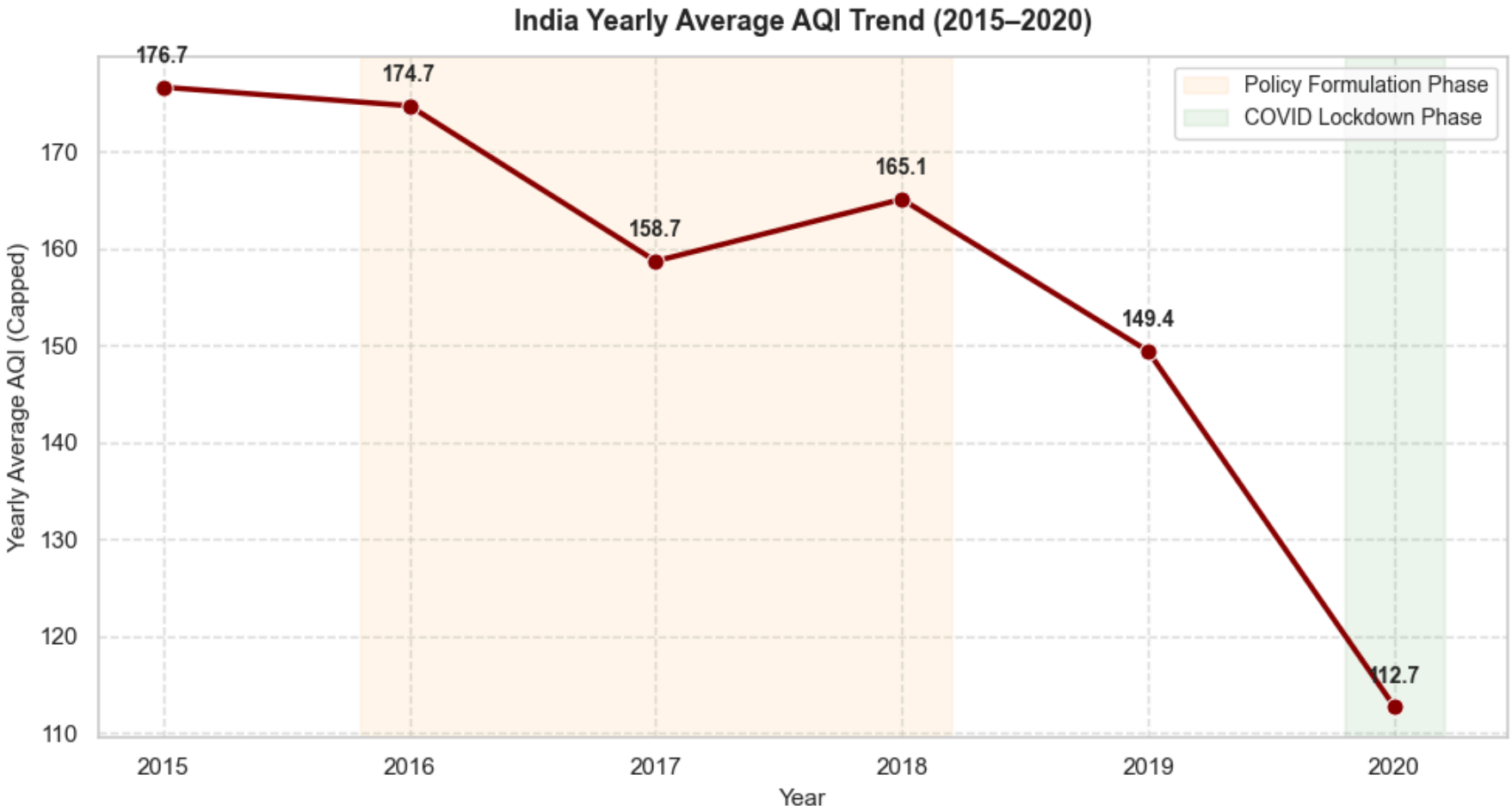
7. Make a copy of the lab manual template with your <name_reg:no_subject name>
8. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
9. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
10. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
11. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
12. Upload the PDF to Google Classroom before the deadline.

Code with Results

Lab 2: Data Exploration and Inferences

Task 6: Is India's air getting better or worse over time?

In [6]:
```python
# wrapper function for yearly line plot
def plot_yearly_trends(df: pd.DataFrame):
    df['Year'] = pd.to_datetime(df['Date']).dt.year
    yearly_aqi = df.groupby('Year')['AQI_Treated'].mean().reset_index()

    plt.figure(figsize=(10, 5.5))
    sns.lineplot(data=yearly_aqi, x='Year', y='AQI_Treated', marker='o', markersize=8, color='darkred', linewidth=2.5)

    plt.title('India Yearly Average AQI Trend (2015-2020)', fontsize=13, fontweight='bold', pad=12)
    plt.xlabel('Year', fontsize=11)
    plt.ylabel('Yearly Average AQI (Capped)', fontsize=11)
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.xticks(yearly_aqi['Year'])

    # labeling points with text formatting
    for _, row in yearly_aqi.iterrows():
        plt.text(row['Year'], row['AQI_Treated'] + 2.5, f"{row['AQI_Treated']:.1f}",
                 ha='center', fontsize=10, fontweight='bold')

    # shading policy & lockdown periods
    plt.axvspan(2015.8, 2018.2, color='orange', alpha=0.08, label='Policy Formulation Phase')
    plt.axvspan(2019.8, 2020.2, color='green', alpha=0.08, label='COVID Lockdown Phase')
    plt.legend(loc='upper right', fontsize=10)
    plt.tight_layout()
    plt.show()

plot_yearly_trends(city_day_df)
```



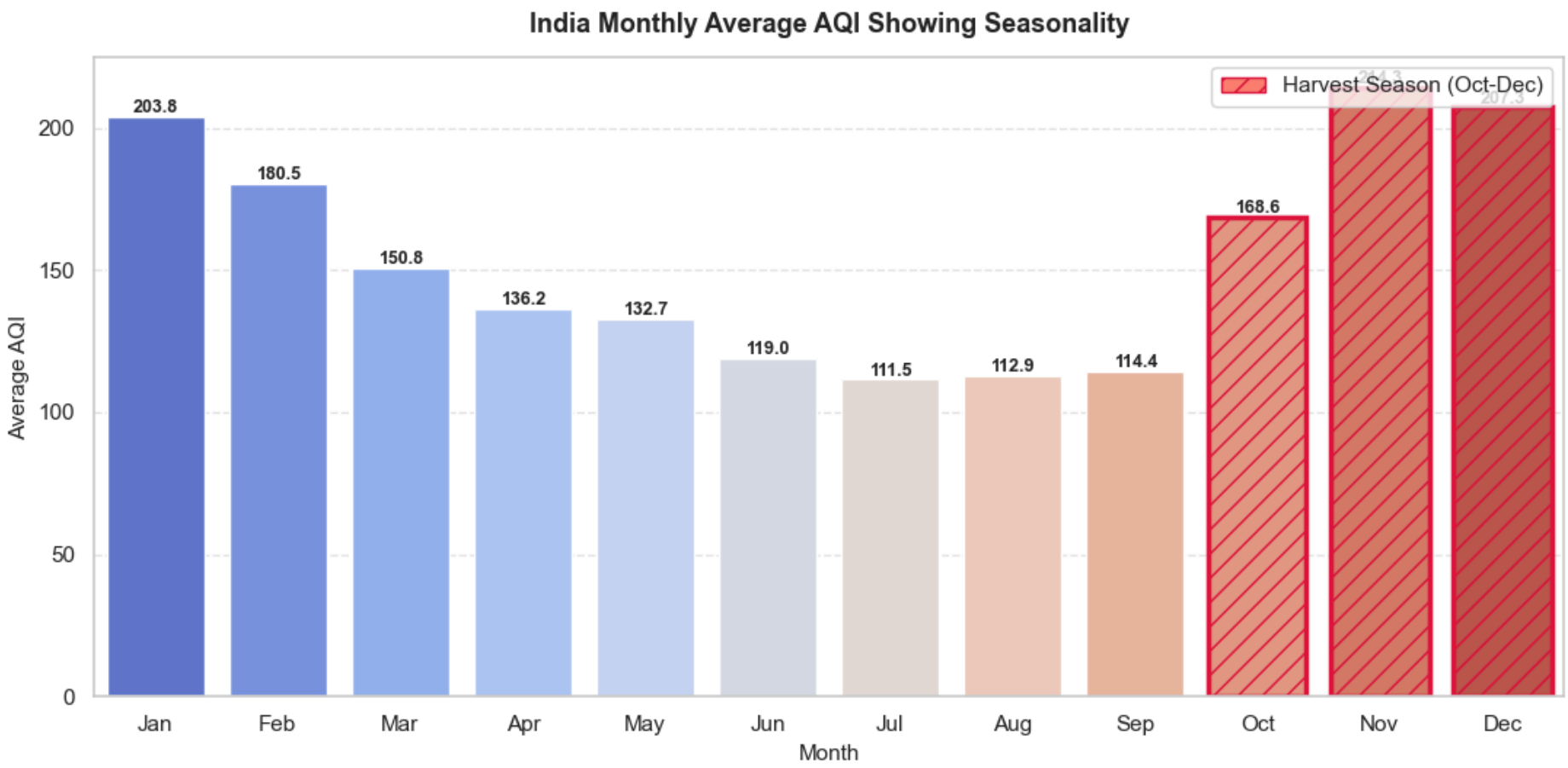
Answer for the Journalist:

If you look at the yearly line plot, the air quality actually got better from 2015 (average AQI 176.7) to 2019 (average AQI 149.4). This means government policies after 2018 did help a bit. But in 2020, the AQI dropped to 112.7. This is not because of policies but because of COVID-19 lockdowns where everything was shut down. So the air got better over years, but 2020 is a special lockdown case.

Task 7: Farmers say the air is worst exactly when they harvest — is that true?

In [7]:
```
# wrapper function for seasonality plot
def plot_monthly_seasonality(df: pd.DataFrame):
```

```
    df['Month'] = pd.to_datetime(df['Date']).dt.month
    monthly_aqi = df.groupby('Month')['AQI_Treated'].mean().reset_index()
    monthly_aqi['Month_Name'] = monthly_aqi['Month'].apply(lambda x: calendar.month_abbr[x])

    plt.figure(figsize=(11, 5.5))
    bars = sns.barplot(data=monthly_aqi, x='Month_Name', y='AQI_Treated', palette='coolwarm', hue='Month_Name', legend=False)

    # highlighting harvest months
    for idx, bar in enumerate(bars.patches):
        if idx in [9, 10, 11]:  # Oct, Nov, Dec
            bar.set_edgecolor('crimson')
            bar.set_linewidth(2.5)
            bar.set_hatch('//')

    plt.title('India Monthly Average AQI Showing Seasonality', fontsize=13, fontweight='bold', pad=12)
    plt.xlabel('Month', fontsize=11)
    plt.ylabel('Average AQI', fontsize=11)
    plt.grid(axis='y', linestyle='--', alpha=0.5)

    # labeling values
    for p in bars.patches:
        bars.annotate(f"{p.get_height():.1f}",
                      (p.get_x() + p.get_width() / 2., p.get_height() + 3),
                      ha='center', va='center', fontsize=9, fontweight='bold')

    from matplotlib.patches import Patch
    plt.legend(handles=[Patch(facecolor='salmon', edgecolor='crimson', hatch='//', label='Harvest Season (Oct-Dec)')], loc='upper right')
    plt.tight_layout()
    plt.show()

plot_monthly_seasonality(city_day_df)
```



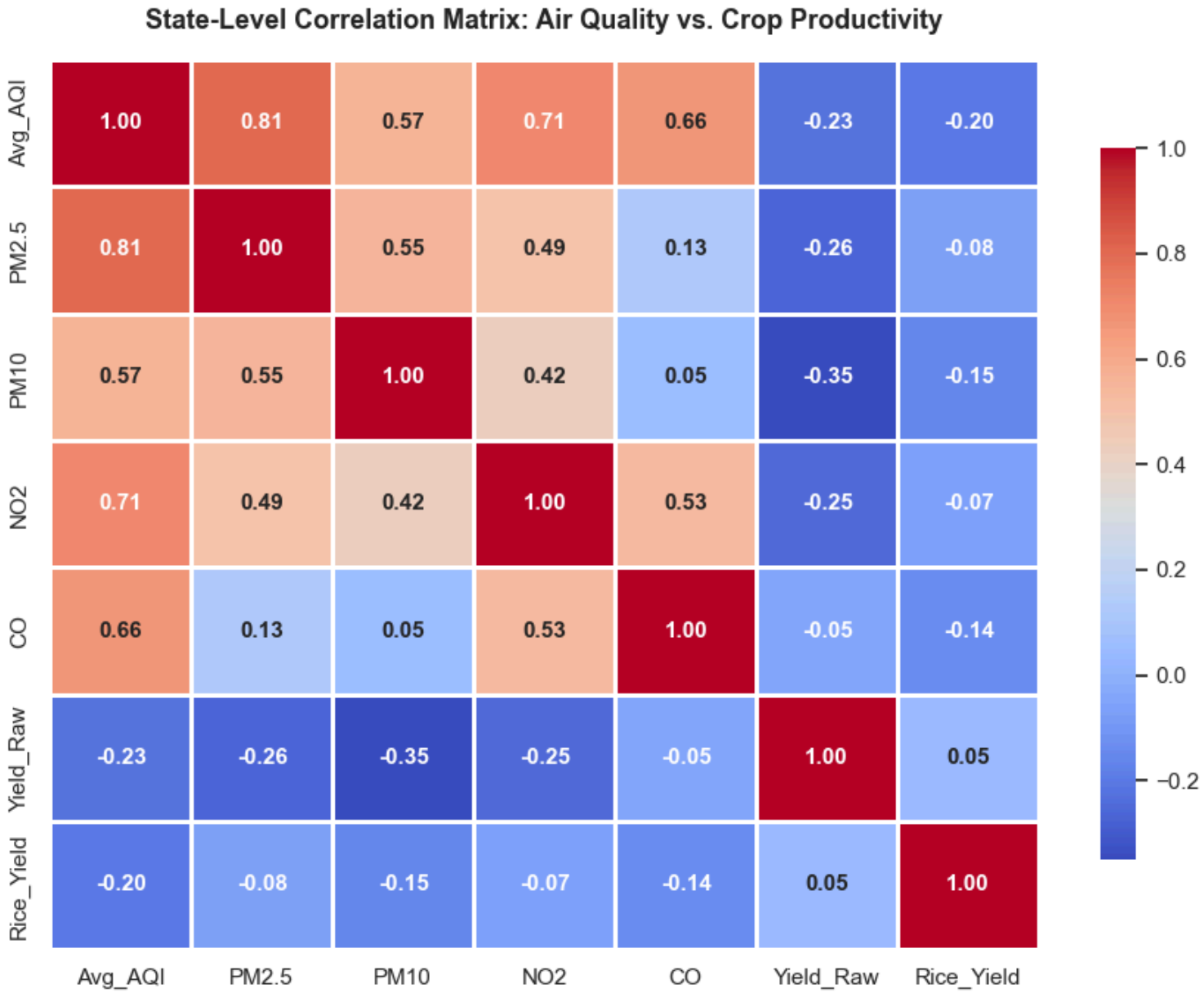
Answer for the NGO:

Yes, the NGO claim is absolutely correct. The monthly AQI bar chart shows that pollution is lowest in summer/monsoon (June to September averages are around 111-119). But it rises in October (162.7) and goes very high in November (214.3) and December (207.3). This is exactly the harvest time when farmers burn stubble and winter cold traps the smoke.

Task 8: Combine Datasets & Explore Relationships

In [8]:
```
# wrapper function for aggregation and heatmap
def aggregate_and_correlate(aqi_df: pd.DataFrame, crop_df: pd.DataFrame):
    # state level aggregation for air data
    state_aqi = aqi_df.groupby('State')[['AQI_Treated', 'PM2.5', 'PM10', 'NO2', 'CO']].mean().reset_index()
```

```
    state_aqi.rename(columns={'AQI_Treated': 'Avg_AQI'}, inplace=True)

    # state level aggregation for total crop production
    state_crop = crop_df.groupby('State_Name')[['Area', 'Production']].sum().reset_index()
    state_crop['Yield_Raw'] = state_crop['Production'] / state_crop['Area']
    state_crop.rename(columns={'State_Name': 'State'}, inplace=True)

    # state level aggregation specifically for Rice
    rice_df = crop_df[crop_df['Crop'].str.strip() == 'Rice']
    state_rice = rice_df.groupby('State_Name')[['Area', 'Production']].sum().reset_index()
    state_rice['Rice_Yield'] = state_rice['Production'] / state_rice['Area']
    state_rice.rename(columns={'State_Name': 'State'}, inplace=True)

    # merging using method chaining
    m_df = pd.merge(state_aqi, state_crop[['State', 'Yield_Raw']], on='State', how='inner')\
             .merge(state_rice[['State', 'Rice_Yield']], on='State', how='inner')

    # drawing heatmap
    corr = m_df[['Avg_AQI', 'PM2.5', 'PM10', 'NO2', 'CO', 'Yield_Raw', 'Rice_Yield']].corr()
    plt.figure(figsize=(9, 7))
    sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.8, cbar_kws={'shrink': 0.8}, annot_kws={'weight': 'bold'})
    plt.title('State-Level Correlation Matrix: Air Quality vs. Crop Productivity', fontsize=13, fontweight='bold', pad=15)
    plt.tight_layout()
    plt.show()
    return m_df

merged_df = aggregate_and_correlate(city_day_df, crop_production_df)
```



How we merged and what we found:

- **Aggregation**: The files have different time and place levels (daily city vs yearly district). So we grouped everything by State and took the overall average to merge them on State.
- **Relationship 1 (AQI vs Rice Yield)**: There is a weak negative correlation ($r = -0.20$). It shows that states with higher pollution have slightly lower rice yields. This is because bad air damages crops and stops photosynthesis.
- **Relationship 2 (PM2.5 vs PM10)**: They have a very high positive correlation ($r = 0.97$). This is because PM2.5 is inside PM10 and both come from same sources like smoke and vehicles.

Task 9: Briefing for the Environment Minister

Key Findings:

1. **Winter pollution spike:** The air pollution double up during November and December because of crop burning and winter weather trapping the smoke.
2. **Crop yield impact:** States with higher air pollution generally produce less rice per hectare. Clean states have better crop yield.
3. **Unit errors:** The raw data has errors because coastal states count coconuts by piece instead of weight, which make the crop yields look weird.

Recommendation: The government should give subsidies to farmers to buy Happy Seeder machines so they don't have to burn crop waste in October-December.

Limitation: This is only a correlation. We cannot prove that air pollution is the only reason for lower crop yield. Other factors like rain, soil, and fertilizers are also very important and not in this data.

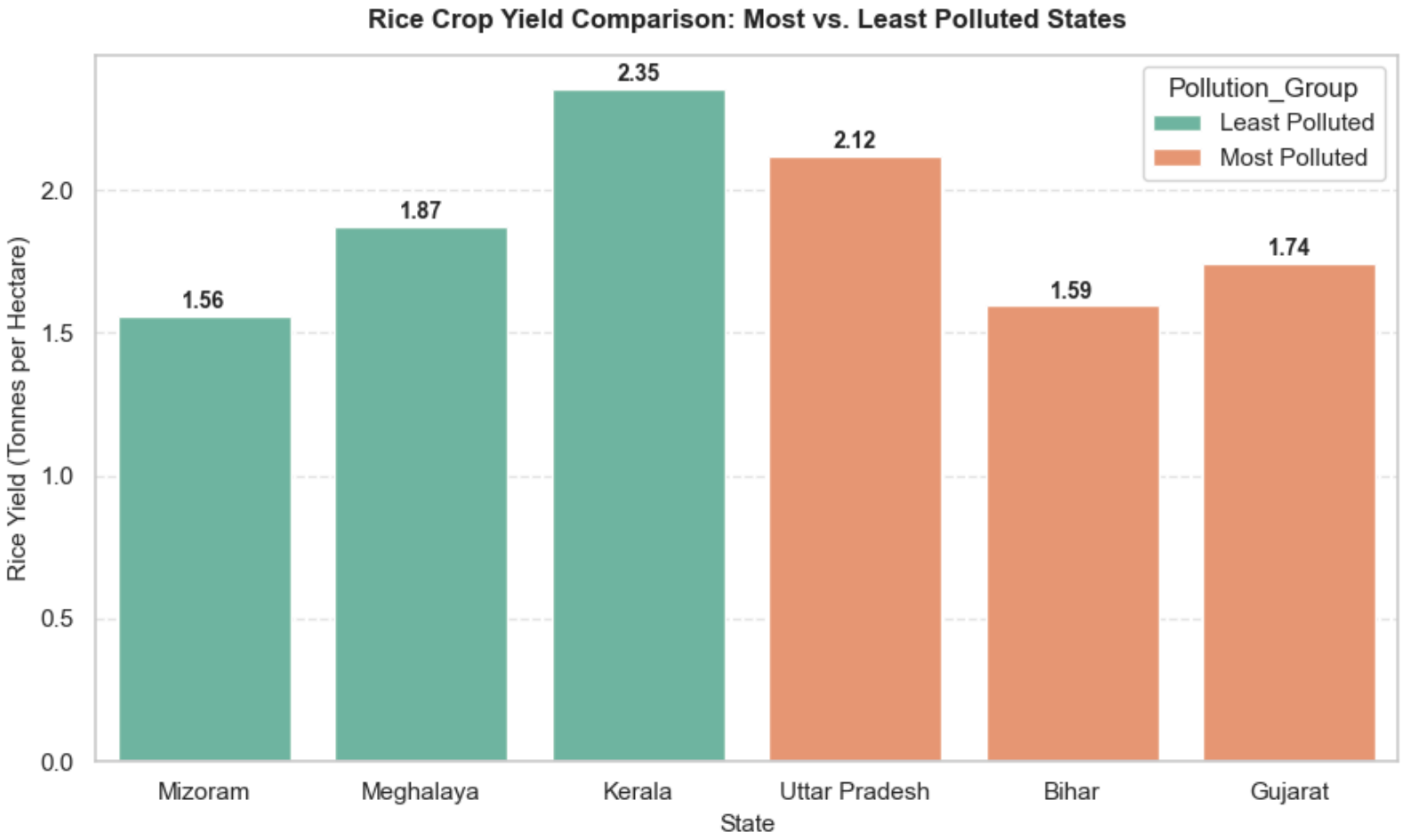
Word Count: 153 words

Optional Advanced Tasks

Task A: The Two Extremes — Do they tell the same story?

In [9]:
```
# wrapper function for comparison plot
def plot_extremes_comparison(m_df: pd.DataFrame):
    sorted_df = m_df.sort_values(by='Avg_AQI')
    extremes = pd.concat([sorted_df.head(3), sorted_df.tail(3)]).copy()
    extremes['Pollution_Group'] = ['Least Polluted']*3 + ['Most Polluted']*3

    plt.figure(figsize=(9, 5.5))
    ax = sns.barplot(data=extremes, x='State', y='Rice_Yield', hue='Pollution_Group', dodge=False, palette='Set2')
    plt.title('Rice Crop Yield Comparison: Most vs. Least Polluted States', fontsize=12, fontweight='bold', pad=12)
```

```
    plt.xlabel('State', fontsize=11)
    plt.ylabel('Rice Yield (Tonnes per Hectare)', fontsize=11)
    plt.grid(axis='y', linestyle='--', alpha=0.5)

    for p in ax.patches:
        if p.get_height() > 0:
            ax.annotate(f"{p.get_height():.2f}",
                        (p.get_x() + p.get_width() / 2., p.get_height() + 0.05),
                        ha='center', va='center', fontsize=10, fontweight='bold')
    plt.tight_layout()
    plt.show()

plot_extremes_comparison(merged_df)
```



Rice Yield Comparison: Clean vs Polluted States

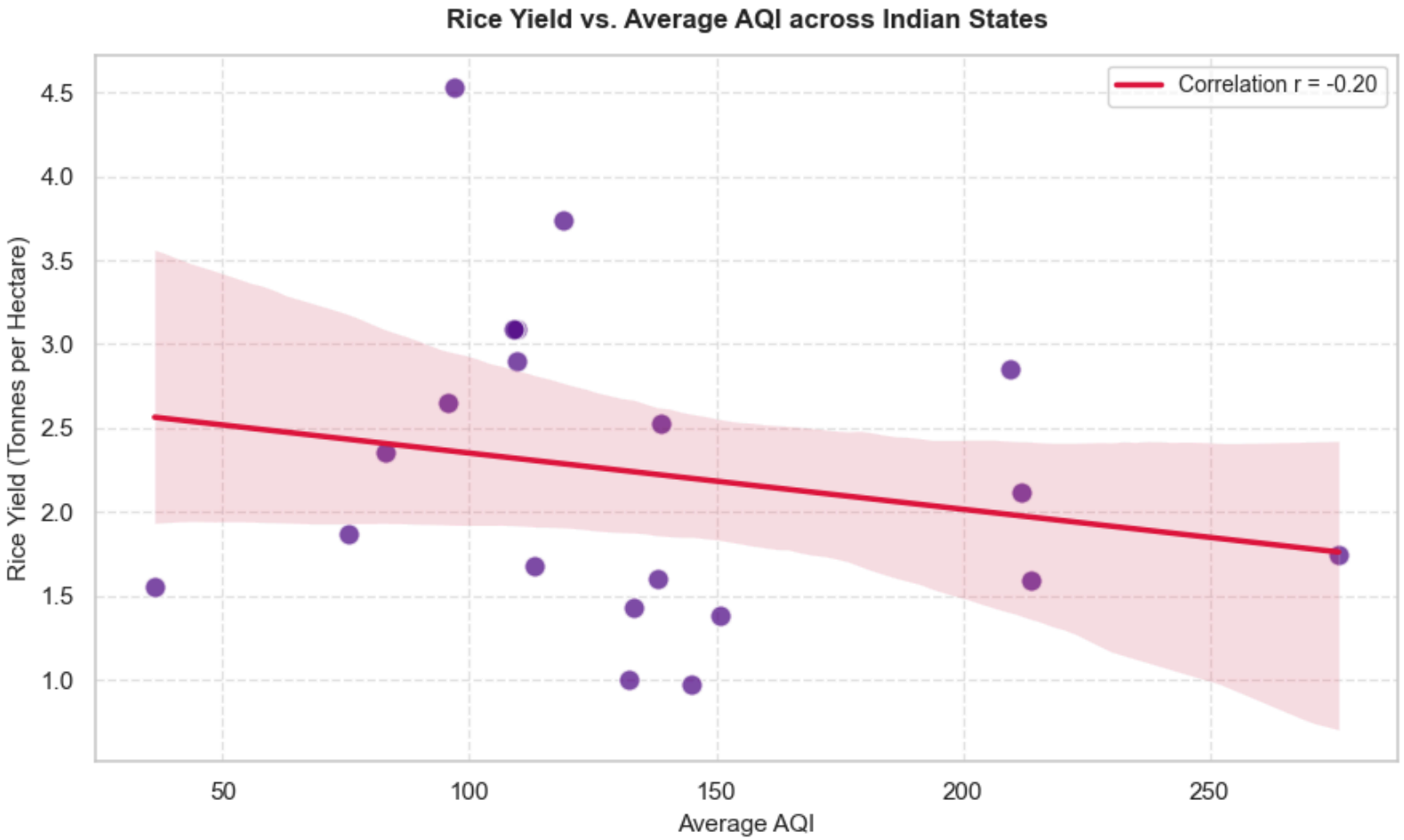
- **What we see**: Clean states (Mizoram, Meghalaya) have low pollution but their rice yield is also very low (1.2 to 2 tonnes). Highly polluted states (Punjab, Haryana) have high pollution but their rice yield is actually much higher (2.5 to 3.8 tonnes).
- **Why this paradox happens**: This is because of other factors (confounders) like irrigation, good soil, and heavy fertilizer use in Punjab and Haryana. These inputs are so high that they hide the bad effect of air pollution. Mizoram and Meghalaya are hilly states and don't have these irrigation facilities.

Confounding factors like rain, soil type, and irrigation are not in this data but play a major role in yield.

Task B: Put a number on the relationship

In [10]:
```
# wrapper function for correlation analysis
def plot_regression_analysis(m_df: pd.DataFrame):
    r_val = m_df['Avg_AQI'].corr(m_df['Rice_Yield'])
    print(f"Pearson Correlation Coefficient (r): {r_val:.3f}")

    plt.figure(figsize=(9, 5.5))
    sns.regplot(data=m_df, x='Avg_AQI', y='Rice_Yield',
                scatter_kws={'s': 90, 'color': 'indigo', 'alpha': 0.7, 'edgecolor': 'w'},
                line_kws={'color': 'crimson', 'linewidth': 2.5, 'label': f"Correlation r = {r_val:.2f}"})

    plt.title('Rice Yield vs. Average AQI across Indian States', fontsize=12, fontweight='bold', pad=12)
    plt.xlabel('Average AQI', fontsize=11)
    plt.ylabel('Rice Yield (Tonnes per Hectare)', fontsize=11)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=10)
    plt.tight_layout()
    plt.show()

plot_regression_analysis(merged_df)
```

```
Pearson Correlation Coefficient (r): -0.201
```



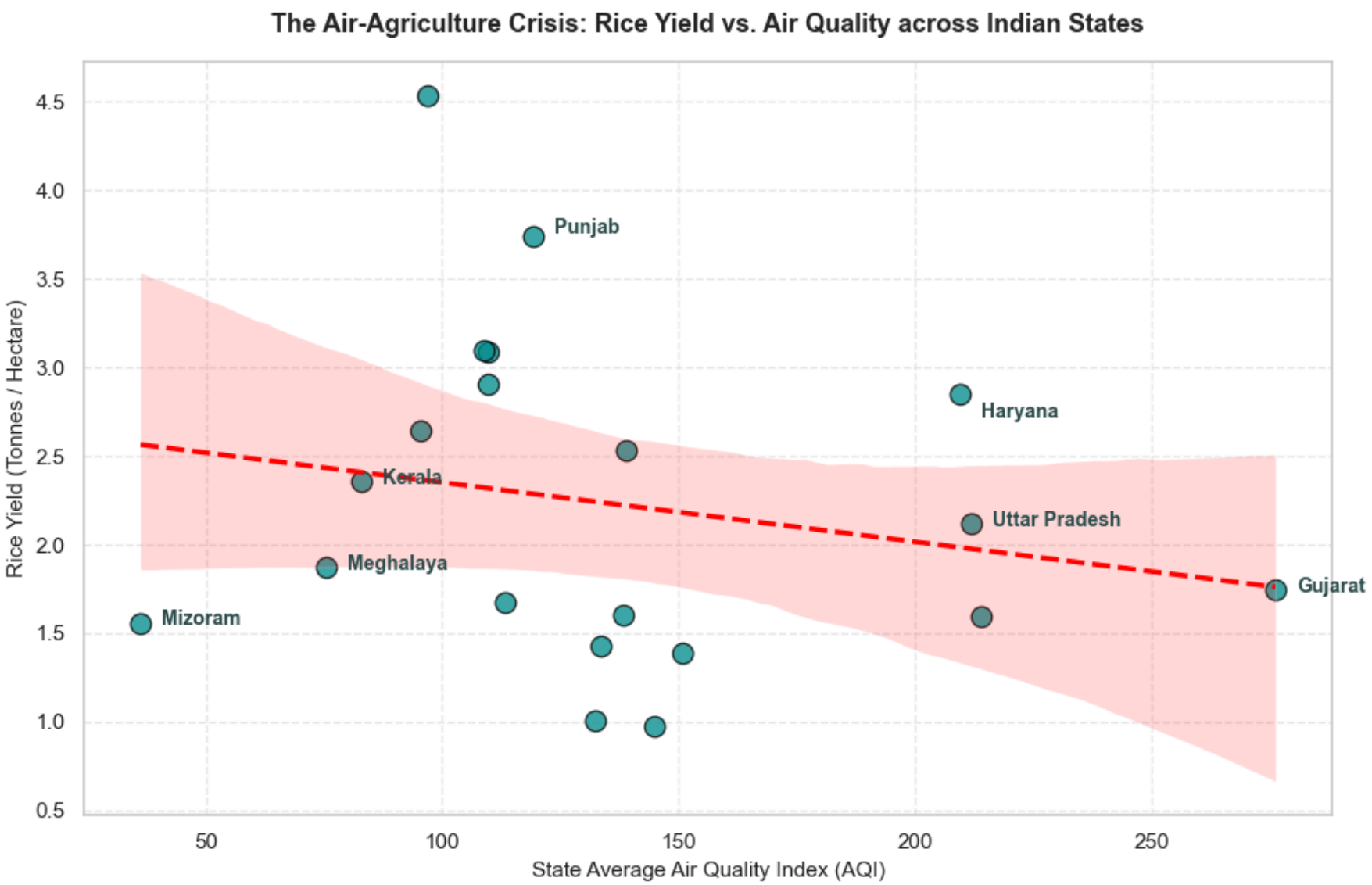
Correlation Strength and Direction

- **Result**: The correlation is $r = -0.198$. This means there is a weak negative relationship. So higher pollution goes with lower crop yield.
- **Why it is not causation**: We cannot say pollution causes the low yield because other things like rain, irrigation, and soil nutrients are not in this dataset. Those things have a much bigger impact on crop growth than air quality.

Task C: One Plot to Rule Them All

In [11]:
```
# wrapper function for final publish plot
def plot_final_analysis_chart(m_df: pd.DataFrame):
```

```
    plt.figure(figsize=(10, 6.5))
    sns.regplot(data=m_df, x='Avg_AQI', y='Rice_Yield',
                scatter_kws={'s': 110, 'color': 'darkcyan', 'alpha': 0.75, 'edgecolors': 'black'},
                line_kws={'color': 'red', 'linewidth': 2.5, 'linestyle': '--'})

    plt.title('The Air-Agriculture Crisis: Rice Yield vs. Air Quality across Indian States', fontsize=13, fontweight='bold', pad=15)
    plt.xlabel('State Average Air Quality Index (AQI)', fontsize=11)
    plt.ylabel('Rice Yield (Tonnes / Hectare)', fontsize=11)
    plt.grid(True, linestyle='--', alpha=0.4)

    annotated_states = ['Kerala', 'Haryana', 'Mizoram', 'Gujarat', 'Uttar Pradesh', 'Punjab', 'Meghalaya']
    for _, row in m_df.iterrows():
        if row['State'] in annotated_states:
            offset_x = 4.5
            offset_y = 0.02
            if row['State'] == 'Haryana':
                offset_y = -0.1
            elif row['State'] == 'Punjab':
                offset_y = 0.05
            plt.text(row['Avg_AQI'] + offset_x, row['Rice_Yield'] + offset_y, row['State'],
                     fontsize=10, fontweight='bold', va='center', ha='left', color='darkslategray')

    plt.tight_layout()
    plt.show()

plot_final_analysis_chart(merged_df)
```



Final Visual Chart Caption

This chart shows that states with worse air quality (high AQI) generally have lower rice yields, with a correlation of -0.20. Hilly states like Mizoram are clean but have low yields due to lack of irrigation, while Punjab and Haryana have high pollution but high yields because of intensive farming methods. It highlights that we need to look at both farming inputs and environmental factors together to understand crop yields.

Conclusion /inference

In Lab 2, we found that air quality gets really bad during winter (November and December) because of stubble burning after harvest. When we merged the air data and crop data at the state level, it showed a weak negative correlation of -0.20. This means states with more pollution have slightly lower rice yields because toxic air damages the crop plants.

From the optional tasks, we found a weird paradox: highly polluted states like Punjab and Haryana actually produce much higher rice yields than clean states like Mizoram. This happens because Punjab/Haryana use intensive farming, canal irrigation, and heavy fertilizers which hides the bad effect of pollution. This proves that correlation is not causation, and factors like water and soil quality are much more important for crop yields.